

Redwan Alexandru Teodor – IT Portfolio y Servicios

Documentación técnica del proyecto

<https://redwan-alexandru-teodor.github.io/>

Redwan Alexandru Teodor
Técnico en Sistemas y Redes
Sevilla, España

6 de octubre de 2026

Resumen

Este documento describe el sitio web personal y profesional de Redwan Alexandru Teodor, técnico en sistemas y redes con base en Sevilla, publicado en <https://redwan-alexandru-teodor.github.io/>. Se trata de una web estática construida con Next.js 16, React 19 y Tailwind CSS 4, que se genera mediante exportación estática y se publica en GitHub Pages a través de GitHub Actions. El sitio presenta el perfil profesional, un portafolio de proyectos y los servicios técnicos ofrecidos en régimen de colaboración. Su formulario de contacto no envía datos a ningún servidor: abre el cliente de correo del visitante con la solicitud ya preparada. El documento recoge el propósito del proyecto, su arquitectura y stack tecnológico, los pasos de instalación y configuración, una guía de uso y las conclusiones con posibles mejoras.

Índice

1. Introducción y propósito	3
1.1. Problema que resuelve	3
1.2. Características clave	3
1.3. Alcance de este documento	3
2. Arquitectura y tecnologías	3
2.1. Stack tecnológico	3
2.2. Dependencias declaradas	4
2.3. Modelo de ejecución: sitio estático	5
2.4. Servicios externos	5
2.5. Estructura de directorios	6
2.6. Flujo de configuración y despliegue	6
3. Guía de instalación y configuración	7
3.1. Requisitos previos	7
3.2. Clonación del repositorio	7
3.3. Instalación de dependencias	7
3.4. Variables de entorno	7
3.4.1. Entorno local	7
3.4.2. Variables de GitHub Actions	8
3.5. Desarrollo y compilación	8
3.6. Despliegue en GitHub Pages	8

4. Manual de uso y guía rápida	9
4.1. Naturaleza de la interfaz	9
4.2. Comandos principales	9
4.3. Navegación del sitio	9
4.4. Uso del formulario de contacto	9
4.5. Mantenimiento habitual del contenido	9
5. Conclusiones y futuras mejoras	10
5.1. Estado actual	10
5.2. Observaciones técnicas	10
5.3. Posibles mejoras	10
5.4. Licencia y uso del código	11

1. Introducción y propósito

1.1. Problema que resuelve

El proyecto es el sitio web personal y profesional de Redwan Alexandru Teodor. Cumple tres funciones: presentar su perfil como técnico en sistemas y redes, exponer el portafolio de proyectos y los servicios técnicos que ofrece, y servir como punto de contacto para solicitudes de soporte. Los servicios se prestan en régimen de colaboración con el objetivo de construir portafolio, sin relaciones contractuales comerciales previas.

1.2. Características clave

- **Diseño moderno y adaptable:** interfaz con Tailwind CSS 4 y colores en OKLCH, responsive para móviles, tablets y escritorio.
- **Modo claro y oscuro:** conmutador en la cabecera que guarda la preferencia en `localStorage` y respeta el tema del sistema.
- **Servicios técnicos:** presentación de los servicios de mantenimiento, redes y sistemas, con una matriz interactiva.
- **Portafolio de proyectos:** galería con visor de imágenes, carrusel táctil y descripciones en Markdown renderizado sin dependencias externas.
- **Contacto sin API de terceros:** el formulario abre el cliente de correo del visitante mediante `mailto:`, con validación por campo e identificador de registro único para cada solicitud.
- **Aviso legal y documentación:** modal con el aviso legal, la política de privacidad (RGPD y LSSI-CE) y un visor del documento en PDF.
- **SEO técnico:** metadatos Open Graph y Twitter Card, datos estructurados JSON-LD, `robots.txt` y `sitemap.xml` generados durante el build.
- **Accesibilidad:** modales con `role="dialog"` y `aria-modal`, cierre con `Escape`, bloqueo de scroll de fondo y respeto a la preferencia de reducir movimiento.
- **Privacidad en el código:** la dirección de contacto no aparece en el repositorio; se inyecta en el build desde una variable de GitHub Actions.

1.3. Alcance de este documento

Este documento se ha elaborado a partir del README del repositorio, el archivo `package.json`, el archivo `next.config.mjs` y el árbol de directorios. Los archivos de componentes, la configuración de `lib/` y el contenido de `data/` se describen según lo que indica el README. El workflow `.github/workflows/deploy.yml` se describe también a partir del README, por lo que cualquier detalle concreto de ese archivo debe verificarse directamente en el repositorio.

2. Arquitectura y tecnologías

2.1. Stack tecnológico

Los rangos con el símbolo `^` indican compatibilidad semántica con la versión indicada, según lo declarado en `package.json`.

Tabla 1: Stack tecnológico del proyecto

Tecnología	Versión	Uso en el proyecto
Node.js	24 (LTS)	Entorno de compilación en el workflow de GitHub Actions
pnpm	pnpm@12.3.4	Gestor de paquetes, fijado en el campo <code>packageManager</code>
Next.js	16.3.3	Framework con App Router, Turbopack y exportación estática
React	^19	Interfaz y estado de los componentes
TypeScript	5.7.3	Tipado estático; los errores de tipo detienen el build
Tailwind CSS	^4.3.3	Estilos con tokens de color en OKLCH (vía <code>@tailwindcss/postcss</code>)
shadcn/ui y Base UI	^4.11.0 / ^1.5.0	Primitivas de interfaz reutilizables en <code>components/ui/</code>
Lucide React	^1.16.0	Iconos en formato SVG
tw-animate-css	^1.4.0	Animaciones de entrada de modales y secciones
clsx, tailwind-merge, class-variance-authority	^2.1.1, ^3.3.1, ^0.7.1	Composición de clases CSS (<code>lib/utils.ts</code>)

2.2. Dependencias declaradas

El archivo `package.json` define los siguientes scripts y dependencias:

```
{
  "name": "inicio",
  "version": "0.1.0",
  "private": true,
  "packageManager": "pnpm@12.3.4",
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "@base-ui/react": "^1.5.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "lucide-react": "^1.16.0",
    "next": "16.3.3",
    "react": "^19",
    "react-dom": "^19",
    "shadcn": "^4.11.0",
    "tailwind-merge": "^3.3.1",
    "tw-animate-css": "^1.4.0"
  }
}
```

```
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4.3.3",
    "@types/node": "^24",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "postcss": "^8.5",
    "tailwindcss": "^4.3.3",
    "typescript": "5.7.3"
  }
}
```

Listing 1: package.json (extracto completo)

2.3. Modelo de ejecución: sitio estático

El proyecto no tiene servidor propio. Next.js genera el sitio completo como archivos estáticos durante el build, y esos archivos se publican en GitHub Pages. La configuración de `next.config.mjs` es la siguiente:

```
const basePath = process.env.NEXT_PUBLIC_BASE_PATH || ''

/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'export',
  // En GitHub Pages el basePath lo aporta el workflow (<nombre-del-repo>),
  // así que sigue funcionando aunque cambies el nombre del repositorio.
  basePath,
  assetPrefix: basePath ? `/${basePath}/` : undefined,
  trailingSlash: true,
  // Los errores de TypeScript hacen fallar el build (y por tanto el despliegue).
  typescript: {
    ignoreBuildErrors: false,
  },
  images: {
    unoptimized: true,
  },
}

export default nextConfig
```

Listing 2: next.config.mjs (extracto)

Puntos relevantes de esta configuración:

- `output: 'export'` genera HTML, CSS y JavaScript estáticos en la carpeta `out/`.
- `basePath` toma su valor de la variable de entorno `NEXT_PUBLIC_BASE_PATH`, que el workflow define como `<nombre-del-repo>`. En local queda vacía.
- `images.unoptimized` está activado porque la exportación estática no dispone de un servidor que optimice las imágenes.

2.4. Servicios externos

- **GitHub Pages:** hosting del sitio estático, servido desde la rama `main`.
- **GitHub Actions:** CI/CD que ejecuta el build y publica el resultado (`.github/workflows/deploy.yml`).

- **Google Search Console:** verificación de propiedad mediante una etiqueta HTML inyectada desde la variable `GOOGLE_SITE_VERIFICATION`.
- **Schema.org (JSON-LD):** datos estructurados para buscadores, incluidos en `app/layout.tsx`.

2.5. Estructura de directorios

```
./
|-- .github/workflows/deploy.yml    # Build y publicación; inyecta BASE_PATH,
    CONTACT_EMAIL, etc.
|-- app/
|   |-- layout.tsx                  # Layout raíz: fuente, metadatos, JSON-LD
|   |-- page.tsx                    # Ensambla las secciones con animación de entrada
|   |-- sitemap.ts, robots.ts      # sitemap.xml y robots.txt generados en el build
|   |-- globals.css                 # Tokens de color y estilos globales
|   |-- not-found.tsx               # Página 404
|-- components/
|   |-- site-header.tsx             # Menú sticky, sección activa, CTA móvil, LinkedIn
    y tema
|   |-- hero.tsx                    # Presentación, disponibilidad y ventajas
|   |-- about.tsx, services.tsx     # Sobre mí y servicios
|   |-- process.tsx                 # Cómo funciona en 3 pasos
|   |-- proyectos.tsx               # Galería, visor con zoom y renderizador Markdown
|   |-- contact.tsx, contact-form.tsx
|   |-- mail-modal.tsx              # Confirmación del botón Enviar correo
|   |-- site-footer.tsx             # Pie, aviso legal y visor del PDF
|   |-- reveal.tsx, back-to-top.tsx # Animación de entrada y botón volver arriba
|   |-- ui/                          # Primitivas reutilizables
|-- data/
|   |-- servicios.ts                 # Servicios y entornos de trabajo (fuente única)
|   |-- proyectos/                  # Un directorio por proyecto
|-- lib/
|   |-- site-config.ts              # basePath, siteUrl, correo de contacto,
    disponibilidad
|   |-- mail.ts                     # ID único y URL mailto:
|   |-- paises.ts                   # Prefijos internacionales y validación telefónica
|   |-- use-scroll-lock.ts          # Bloqueo de scroll compartido por los modales
|   |-- use-active-section.ts       # Sección visible (resalta el menú)
|   |-- use-theme.ts                # Tema claro/oscuro compartido
|   |-- utils.ts                    # Función cn()
|-- public/                          # Activos estáticos servidos tal cual
    |-- .nojekyll, icon.svg, og_image.png
    |-- images/perfil.jpg           # Foto de perfil
    |-- docs/Documentacion.pdf      # Documentación del proyecto
    |-- proyectos/<slug>/            # Imágenes de cada proyecto
```

Listing 3: Árbol de directorios del repositorio

2.6. Flujo de configuración y despliegue

La Figura 1 resume cómo llegan los valores de configuración al sitio final. Las variables definidas en GitHub se inyectan en el build, y el resultado estático se publica en GitHub Pages.

Las dependencias entre módulos son las siguientes:

- `data/servicios.ts` alimenta `services.tsx` y el JSON-LD de `layout.tsx`.
- `data/proyectos/` alimenta `proyectos.tsx` y `sitemap.ts`.

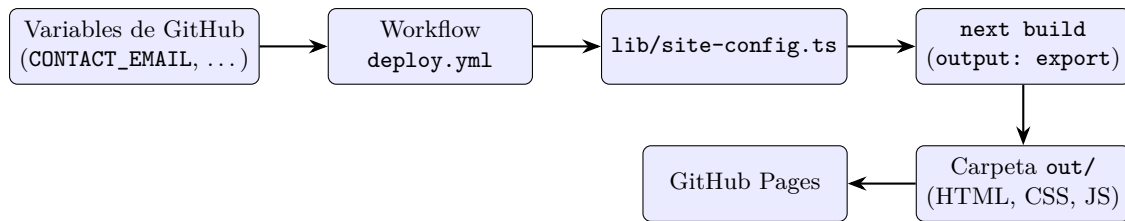


Figura 1: Flujo de configuración y despliegue estático.

- `lib/mail.ts` es utilizado por `mail-modal.tsx` y `contact-form.tsx`.
- `lib/use-scroll-lock.ts` lo comparten todos los modales.
- Los activos binarios (imágenes y PDF) se guardan siempre en `public/`, porque la exportación estática solo copia esa carpeta.

3. Guía de instalación y configuración

3.1. Requisitos previos

- **Node.js 24 (LTS):** es la versión que usa el workflow de despliegue.
- **pnpm:** en la versión fijada en el campo `packageManager` (`pnpm@12.3.4`). Con Corepack activado, se usa esa versión de forma automática.
- **Git:** para clonar el repositorio.
- **Cuenta de GitHub** con permisos sobre el repositorio, necesaria solo para configurar variables y publicar en GitHub Pages.
- **Opcional:** propiedad verificada en Google Search Console.

3.2. Clonación del repositorio

```
git clone https://github.com/Redwan-Alexandru-Teodor/redwan-alexandru-teodor.github.io
.git
cd redwan-alexandru-teodor.github.io
```

Listing 4: Clonar el repositorio

3.3. Instalación de dependencias

```
corepack enable
pnpm install
```

Listing 5: Preparar pnpm e instalar dependencias

El archivo `pnpm-lock.yaml` fija las versiones exactas. Conviene instalar siempre con `pnpm install` para respetarlas.

3.4. Variables de entorno

3.4.1. Entorno local

Para probar el formulario de correo en local, crea un archivo `.env.local` en la raíz del proyecto. Git lo ignora por defecto.

```
NEXT_PUBLIC_CONTACT_EMAIL=tu@correo.com
```

Listing 6: Archivo .env.local

3.4.2. Variables de GitHub Actions

Estas variables se configuran en *Settings* → *Secrets and variables* → *Actions*, a nivel de repositorio (no de entorno). Sirven tanto *Variables* como *Secrets*. Tras crearlas o modificarlas hay que volver a desplegar.

Tabla 2: Variables de configuración del despliegue

Nombre	Finalidad	Si falta
CONTACT_EMAIL	Destinatario de los correos del formulario. También se acepta NEXT_PUBLIC_CONTACT_EMAIL.	Botones y formulario de correo desactivados
GOOGLE_SITE_VERIFICATION	Valor de la etiqueta HTML de Google Search Console (opcional).	No se añade la etiqueta

Las variables con prefijo NEXT_PUBLIC_ se incrustan en el código durante el build, por lo que cualquier visitante puede verlas en el sitio publicado. Por eso el correo de contacto se mantiene fuera del repositorio.

3.5. Desarrollo y compilación

```
# Servidor de desarrollo con recarga en caliente (http://localhost:3000)
pnpm dev

# Compilación de producción: genera el sitio estático en out/
pnpm build
```

Listing 7: Comandos de desarrollo y build

3.6. Despliegue en GitHub Pages

1. Configura las variables de GitHub Actions descritas en la sección anterior.
2. Sube los cambios a la rama `main`.
3. Para desplegar manualmente, ve a *Actions* → *Deploy to GitHub Pages* → *Run workflow*.
4. **Google Search Console:** añade una propiedad de tipo *Prefijo de URL* con la dirección <https://redwan-alexandru-teodor.github.io/>, verifícala con la etiqueta HTML (el valor va en `GOOGLE_SITE_VERIFICATION`), pulsa *Verificar* tras el despliegue y envía `sitemap.xml` en el apartado *Sitemaps*.

```
pnpm build
python3 -m http.server 8000 --directory out
# Abrir http://localhost:8000
```

Listing 8: Vista previa de la versión estática (alternativa a next start)

4. Manual de uso y guía rápida

4.1. Naturaleza de la interfaz

El proyecto es un sitio web público. No expone una API ni endpoints propios: toda la interacción ocurre en el navegador. Las únicas integraciones externas son el cliente de correo del visitante (mediante `mailto:`) y la verificación de Search Console.

4.2. Comandos principales

Tabla 3: Comandos npm/pnpm disponibles en `package.json`

Comando	Descripción
<code>pnpm install</code>	Instala las dependencias fijadas en <code>pnpm-lock.yaml</code> .
<code>pnpm dev</code>	Arranca el servidor de desarrollo con recarga en caliente en <code>http://localhost:3000</code> .
<code>pnpm build</code>	Compila el sitio y genera la versión estática en <code>out/</code> . Los errores de TypeScript interrumpen el proceso.
<code>pnpm start</code>	Definido en <code>package.json</code> , pero <code>next start</code> no está pensado para la exportación estática. Para previsualizar <code>out/</code> se recomienda un servidor estático.

4.3. Navegación del sitio

- **Cabecera fija:** menú que resalta la sección visible, botón de contacto siempre accesible en móvil, enlace a LinkedIn y conmutador de tema claro u oscuro.
- **Hero:** presentación, disponibilidad y ventajas del servicio.
- **Sobre mí y Servicios:** perfil profesional y oferta de servicios, con un índice de tecnologías.
- **Cómo funciona:** proceso de colaboración en tres pasos.
- **Proyectos:** galería con visor de imágenes con zoom y descripciones en Markdown.
- **Contacto:** formulario de solicitud de soporte.
- **Pie de página:** enlaces de información legal, que abren el aviso legal y el visor del PDF de documentación. Incluye botón para volver arriba.

4.4. Uso del formulario de contacto

1. Rellena nombre, email, servicio, asunto y mensaje. Cada campo se valida al instante y muestra su error junto al campo.
2. Lee las condiciones hasta el final y acéptalas. La aceptación se recuerda durante la sesión.
3. Pulsa «Preparar mensaje». Se abre un modal con dos opciones.
4. Elige «Enviar desde mi aplicación de correo», que abre el cliente de correo con el asunto y el cuerpo ya rellenos, o «Copiar el mensaje», que deja cada parte lista para pegar.
5. La pantalla final recuerda que el correo no se envía automáticamente y permite limpiar el formulario.

4.5. Mantenimiento habitual del contenido

- **Servicios:** edita `data/servicios.ts`, que es la fuente única de los servicios y entornos de trabajo.

- **Proyectos:** crea un directorio nuevo en `data/proyectos/` y sus imágenes en `public/proyectos/[slug]/`. Para el formato exacto de los archivos, consulta un proyecto existente.
- **Foto de perfil:** sustituye `public/images/perfil.jpg`.
- **Documentación en PDF:** sustituye `public/docs/Documentacion.pdf`. En iPhone, Safari no muestra PDF de forma fiable dentro de un `iframe`, por eso el modal incluye siempre un enlace para abrir o descargar el archivo.
- **Disponibilidad:** cambia el valor de `disponible` en `lib/site-config.ts`.
- **Aviso legal:** su texto completo está en `components/site-footer.tsx`.

5. Conclusiones y futuras mejoras

5.1. Estado actual

El proyecto es un sitio web estático, funcional y publicado en producción. Su arquitectura es deliberadamente sencilla: no tiene backend, la configuración sensible se gestiona con variables de GitHub Actions y el envío de solicitudes se delega en el cliente de correo del visitante. Esta elección reduce la superficie de ataque, elimina costes de servidor y evita almacenar datos personales en sistemas propios. A cambio, el formulario no garantiza la entrega de la solicitud, que depende de que el visitante envíe el correo desde su aplicación.

5.2. Observaciones técnicas

Durante el análisis se han detectado algunas inconsistencias que conviene revisar:

- **Nombre del repositorio:** el README indicaba `github.com/Redwan-Alexandru-Teodor/inicio`, pero el repositorio real se llama `redwan-alexandru-teodor.github.io`. El campo `name` de `package.json` también es `inicio`.
- **Ruta del PDF:** el árbol de directorios sitúa el documento en `public/docs/`, mientras que el apartado de contenido legal lo menciona como `public/Documentacion.pdf`. Hay que confirmar cuál es la ruta correcta.
- **Script start:** `next start` no es compatible con `output: 'export'`. Sería más claro eliminarlo o sustituirlo por un servidor estático.
- **Optimización de imágenes:** `images.unoptimized` está activado, así que las imágenes de `next/image` no se optimizan. Es una limitación inherente a la exportación estática, pero conviene tenerla en cuenta al añadir fotos de gran tamaño.
- **Pruebas y calidad:** `package.json` no define scripts de pruebas ni de lint. El control de calidad depende del tipado de TypeScript durante el build.

5.3. Posibles mejoras

- Añadir pruebas unitarias para la validación del formulario (`lib/mail.ts`, `lib/paises.ts`) y pruebas end-to-end del flujo de los modales.
- Incluir en el workflow de GitHub Actions un paso de lint y de comprobación de tipos en las pull requests, antes del despliegue.
- Unificar el nombre del repositorio, el campo `name` y las rutas documentadas en el README.

- Generar automáticamente la vista previa del sitio en cada pull request para revisar cambios visuales antes de publicarlos.
- Añadir comprobaciones de rendimiento y accesibilidad, por ejemplo con Lighthouse CI, para vigilar la regresión de métricas entre versiones.
- Ofrecer una versión en inglés del sitio, ampliando el alcance a clientes internacionales, con el sistema de rutas de Next.js por idioma.
- Valorar un servicio de formularios que cumpla el RGPD, si en el futuro se quiere garantizar la entrega de las solicitudes sin depender del cliente de correo del visitante.

5.4. Licencia y uso del código

El código se publica como referencia del trabajo de su autor. El contenido está protegido por derechos de autor: © 2026 Redwan Alexandru Teodor. Todos los derechos reservados. Para reutilizar, adaptar o usar el código en otro proyecto, es necesario contactar previamente con el autor.